YNOV CAMPUS / JOUR 4

Jour 4 Protéger le travail concurrent

Transactions, MVCC, verrous et récupération. Apprenez avec des exemples commentés, puis exercez-vous dans TP04.

PostgreSQL 18 | 2e année de bachelor | Édition pédagogique progressive

Une transaction rassemble les modifications liées

Une transaction est un groupe d’opérations de base de données traité comme une unité de travail. Imaginez transférer 20 € d’Alice à Bilal : soustraire à Alice sans ajouter à Bilal ferait perdre de l’argent. CampusMarket rencontre le même problème lorsqu’un passage en caisse réserve plusieurs produits. Il faut que toutes les réservations requises réussissent ou qu’aucune ne subsiste.

Résultat / données d’exemple

Résultat	Alice (€)	Bilal (€)	Total (€)
Avant le transfert	100	40	140
Transfert complet	80	60	140
Demi-transfert défectueux	80	40	120

Sens des mots-clés

transaction — Groupe d’opérations acceptées ou annulées ensemble.

unité de travail — Action métier complète, comme la réservation d’un panier.

ACID désigne quatre garanties distinctes

ACID abrège quatre propriétés des transactions : atomicité, cohérence, isolement et durabilité. Chacune répond à une question différente sur notre passage en caisse. L’atomicité regroupe les modifications ; la cohérence dépend de règles et contraintes correctes ; l’isolement contrôle les interactions avec les autres travaux ; la durabilité préserve une validation confirmée selon les paramètres de durabilité de la base. ACID ne corrige pas automatiquement une réservation mal écrite.

Résultat / données d’exemple

Propriété	Exemple CampusMarket
Atomicité	Le produit 2 ne peut pas être réservé : annuler aussi le produit 1.
Cohérence	CHECK (stock >= 0) rejette un stock négatif.
Isolement	Un autre passage en caisse ne peut pas lire notre modification de stock inachevée.
Durabilité	Après une validation durable réussie, la réservation survit à un redémarrage.

ACID désigne quatre garanties distinctes / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

atomicité — Toutes les modifications de la transaction subsistent, ou aucune.

cohérence — Les contraintes déclarées et les règles correctement implémentées restent satisfaites.

isolement — Règles d’interaction entre transactions simultanées.

durabilité — Une validation confirmée est conservée selon la configuration de durabilité.

Préparer cinq unités sans modifier le jeu de données du cours

Nos expériences utilisent une table jetable du schéma work. Chaque ligne représente un produit et sa quantité disponible. Commencez avec cinq unités du produit 1 et cinq du produit 2, comme dans l’exercice de panier TP04. Exécutez cette préparation une fois, lorsque les deux connexions sont hors transaction. Ne réutilisez la réinitialisation qu’entre les expériences, après validation ou annulation sur les deux connexions.

SQL / lire l’instruction une clause à la fois

```sql
CREATE SCHEMA IF NOT EXISTS work;
CREATE TABLE IF NOT EXISTS work.stock_demo (
  product_id integer PRIMARY KEY,
  stock integer NOT NULL CHECK (stock >= 0)
);
TRUNCATE work.stock_demo;
INSERT INTO work.stock_demo VALUES (1, 5), (2, 5);
```

Sens des mots-clés

schéma — Conteneur nommé de tables, ici work.

PRIMARY KEY — Exige un identifiant de produit unique et non NULL.

CHECK — Rejette une ligne qui viole la condition écrite.

TRUNCATE — Supprime toutes les lignes de cette table jetable.

BEGIN démarre l’unité ; COMMIT ou ROLLBACK la termine

BEGIN ouvre une transaction explicite. UPDATE fait passer le produit 1 de cinq unités à une et RETURNING affiche la ligne modifiée par cette instruction. COMMIT accepterait la transaction ; ROLLBACK annule ses modifications. La valeur affichée montre ce que notre connexion voit actuellement, pas qu’une validation a eu lieu. Exécutez cette version pour vous entraîner à annuler une réservation d’essai en sécurité.

SQL / lire l’instruction une clause à la fois

```sql
BEGIN;
UPDATE work.stock_demo
SET stock = stock - 4
WHERE product_id = 1
RETURNING product_id, stock;
ROLLBACK;
SELECT product_id, stock
FROM work.stock_demo ORDER BY product_id;
```

Sens des mots-clés

BEGIN — Démarre un bloc de transaction explicite.

COMMIT — Accepte les modifications de la transaction.

ROLLBACK — Annule les modifications de la transaction.

RETURNING — Renvoie les lignes affectées par une instruction d’écriture.

Une ligne renvoyée peut encore être annulée

L’UPDATE de l’exemple précédent renvoie stock à 1, car cinq moins quatre font un. Après ROLLBACK, le SELECT final affiche de nouveau les deux quantités initiales. Si nous réexécutons l’exemple depuis l’état réinitialisé et remplaçons ROLLBACK par COMMIT, le produit 1 reste à une unité. Ce sont deux exécutions possibles : ne combinez jamais leurs résultats comme si une seule exécution avait produit les deux fins.

Résultat / données d’exemple

Moment ou variante	Produit 1	Produit 2
Réinitialisation avant BEGIN	5	5
Notre état après UPDATE	1	5
État final après le ROLLBACK montré	5	5
Autre exécution se terminant par COMMIT	1	5

Sens des mots-clés

sortie d’instruction — Lignes qu’une commande unique renvoie au client.

modification validée — Modification acceptée par une transaction achevée avec succès.

SAVEPOINT crée un point d’annulation partielle

Un point de sauvegarde marque une position dans une transaction ouverte. ROLLBACK TO SAVEPOINT annule les modifications effectuées après cette position et conserve les précédentes. Ici, nous conservons une réservation d’essai d’une unité du produit 1, mais annulons la réservation facultative de deux unités du produit 2. Le COMMIT final n’accepte que la première réservation. Un point de sauvegarde ne valide rien à lui seul.

SQL / lire l’instruction une clause à la fois

```sql
BEGIN;
UPDATE work.stock_demo SET stock = stock - 1
WHERE product_id = 1;
SAVEPOINT before_optional;
UPDATE work.stock_demo SET stock = stock - 2
WHERE product_id = 2;
ROLLBACK TO SAVEPOINT before_optional;
COMMIT;
```

Sens des mots-clés

SAVEPOINT — Nomme un point d’annulation dans la transaction courante.

ROLLBACK TO SAVEPOINT — Annule le travail après un point nommé et conserve le travail antérieur.

L’annulation partielle ne conserve que le travail avant le repère

Le premier UPDATE précède le point de sauvegarde : le produit 1 reste donc à quatre après l’annulation partielle. Le second UPDATE le suit : le produit 2 repasse donc de trois à cinq. COMMIT accepte alors l’état subsistant. Si une instruction provoque une erreur, la transaction refuse généralement tout travail ultérieur jusqu’à ROLLBACK ou jusqu’au retour à un point de sauvegarde établi avant l’erreur.

Résultat / données d’exemple

Étape	Produit 1	Produit 2
Réinitialisation	5	5
Réserver l’unité obligatoire	4	5
Réserver les deux unités facultatives	4	3
ROLLBACK TO SAVEPOINT	4	5
COMMIT	4	5

L’annulation partielle ne conserve que le travail avant le repère / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

annulation partielle — Annulation des modifications récentes et conservation des précédentes dans la même transaction.

transaction en échec — Transaction ayant rencontré une erreur, qui ne peut continuer normalement avant récupération.

Exercice 1 : décider si tout le panier réussit

Réinitialisez les deux produits à cinq unités. Un client veut quatre unités du produit 1 et six unités du produit 2 et n’accepte le panier que si les deux réservations réussissent. Écrivez une transaction avec les commandes connues UPDATE, WHERE, RETURNING et ROLLBACK. Prévoyez d’abord les deux jeux de lignes renvoyés. Décidez ensuite quelle fin de transaction préserve la règle du client et expliquez votre choix.

Sens des mots-clés

garde de réservation — Condition n’autorisant une modification que si le stock suffit.

règle du panier — Toutes les réservations de produits demandées réussissent ensemble.

Suivez le raisonnement

1. Lancez BEGIN et essayez le produit 1 avec stock >= 4.
2. Essayez le produit 2 avec stock >= 6 et examinez RETURNING.
3. Choisissez COMMIT ou ROLLBACK pour le panier complet.

Corrigé expliqué : l’échec de la deuxième réservation annule la première

Le premier UPDATE protégé renvoie le produit 1 avec stock à 1. Le second UPDATE ne renvoie aucune ligne, car cinq unités ne peuvent satisfaire une demande de six. Cet échec n’exige aucune erreur SQL : la condition WHERE ne correspond simplement à aucune ligne. Comme le panier exige les deux produits, l’application choisit ROLLBACK. Valider la première réservation seule violerait la règle du client énoncée.

SQL / lire l’instruction une clause à la fois

```sql
BEGIN;
UPDATE work.stock_demo SET stock = stock - 4
WHERE product_id = 1 AND stock >= 4
RETURNING product_id, stock;
UPDATE work.stock_demo SET stock = stock - 6
WHERE product_id = 2 AND stock >= 6
RETURNING product_id, stock;
ROLLBACK;
```

Sens des mots-clés

résultat sans ligne — Aucune ligne n’a satisfait la condition d’écriture, donc aucune n’a été modifiée.

décision de l’application — Choix métier fait après examen des résultats de la base de données.

Preuve du panier : un échec métier peut être du SQL valide

La deuxième instruction a réussi au sens SQL, mais n’a pas réservé le produit demandé. Le tableau des résultats distingue ces sens. Après ROLLBACK, la quantité totale disponible est à nouveau dix. C’est pourquoi la seule gestion des erreurs ne suffit pas à décider si le passage en caisse a réussi : les applications doivent aussi examiner des résultats significatifs, comme les identifiants et quantités de produits renvoyés, avant de valider la transaction.

Résultat / données d’exemple

Élément probant	Produit 1	Produit 2	Interprétation
UPDATE RETURNING	1	Aucune ligne	Deuxième réservation échouée
Avant ROLLBACK dans notre session	1	5	Modification partielle encore ouverte
SELECT final après ROLLBACK	5	5	Aucune réservation du panier ne subsiste

Sens des mots-clés

réussite SQL — Commande achevée sans erreur de base de données.

réussite métier — L’action complète a satisfait la règle du client.

Deux connexions permettent d’observer le travail concurrent

Une connexion est un échange d’un client avec PostgreSQL. Une session est le contexte côté serveur de cette connexion, comprenant sa transaction ouverte et ses paramètres. Le travail concurrent signifie que deux sessions ont des transactions ouvertes sur des périodes qui se chevauchent. Ouvrez deux connexions distinctes à la même base dans l’outil de requête et nommez-les A et B. Un second onglet d’éditeur n’est utile que s’il utilise réellement une autre connexion.

DÉMONSTRATION INTERACTIVE / suivez l’ordre des sessions

```
SELECT current_database() AS database_name,
       pg_backend_pid() AS session_pid;
```

Suivez le raisonnement

1. SESSION A : ouvrez la première connexion et exécutez cette requête avant BEGIN.

2. SESSION B : ouvrez une autre connexion à la même base et exécutez la même requête.

3. État attendu : les noms de base correspondent, les identifiants de processus diffèrent et aucune session n’a de transaction ouverte.

Sens des mots-clés

connexion — Échange d’un client avec le serveur de base de données.

session — État associé à une connexion.

concurrent — Dont les périodes se chevauchent, même sans frappes simultanées.

pg_backend_pid() — Renvoie l’identifiant du processus serveur courant.

La session A voit sa propre modification inachevée

Réinitialisez le produit 1 à cinq lorsque les deux sessions sont hors transaction. Dans A, démarrez une transaction et réservez quatre unités sans la terminer. RETURNING montre à A qu'il reste une unité. La modification est non validée : A ne l'a pas acceptée avec COMMIT. Laissez A ouverte et passez à B seulement après réception du résultat par A ; l'ordre de ces actions compte.

DÉMONSTRATION INTERACTIVE / suivez l'ordre des sessions

```
BEGIN;
UPDATE work.stock_demo SET stock = stock - 4
WHERE product_id = 1
RETURNING stock;
```

Sens des mots-clés

non validé — Modifié dans une transaction qui n'a pas été validée avec succès.

écritures propres — Modifications déjà effectuées par la même transaction.

Suivez le raisonnement

1. SESSION A, étape 1 : exécutez les commandes affichées et faites une pause ; RETURNING vaut 1.
2. SESSION B, ensuite : ne modifiez pas les données ; passez à la diapositive suivante.
3. État attendu : la valeur validée du produit 1 reste 5, tandis que A voit sa propre valeur 1.

Un lecteur ordinaire voit le stock validé, puis une validation ultérieure

Pendant que A reste ouverte, B exécute un SELECT ordinaire et voit cinq unités. B ne voit pas la valeur inachevée de un de A. Ensuite, A exécute COMMIT. Lorsque B lance un nouveau SELECT sous le niveau d’isolement par défaut, B voit un. « Validé » signifie accepté par l’auteur de l’écriture ; « visible » signifie inclus dans la vue d’un lecteur donné. Ces notions sont liées, mais ne sont pas identiques.

Résultat / données d’exemple

Observation chronologique	A voit	B voit
A a modifié ; B lit	1	5
A a validé ; B lance une nouvelle lecture	1	1

Sens des mots-clés

validé — Accepté par une transaction ayant achevé COMMIT avec succès.

visible — Inclus dans la vue de la base utilisée par une requête donnée.

Un lecteur ordinaire voit le stock validé, puis une validation ultérieure / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. SESSION B, étape 2 : SELECT stock FROM work.stock_demo WHERE product_id = 1; valeur attendue : 5.
2. SESSION A, étape 3 : exécutez COMMIT ; sa réservation est acceptée.
3. SESSION B, étape 4 : réexécutez ce SELECT ; valeur attendue : 1. Les deux sessions sont maintenant hors transaction.

MVCC donne un instantané aux lectures normales

PostgreSQL utilise le contrôle de concurrence multiversion, abrégé MVCC : il conserve des versions des lignes pour qu’un lecteur utilise une vue validée adaptée pendant qu’une autre session écrit. Un instantané est l’ensemble des versions validées des lignes visibles par une requête ou transaction, plus les modifications propres à cette transaction. Un SELECT ordinaire lit généralement une version visible sans attendre la fin de l’écriture d’une ligne.

Résultat / données d’exemple

Opération pendant la modification inachevée de la ligne par A	Comportement habituel dans cette leçon
B : SELECT ordinaire	Lit l’ancienne valeur validée du stock
B : UPDATE de la même ligne	Attend la fin de A
A : SELECT de sa ligne modifiée	Lit sa propre valeur inachevée

Sens des mots-clés

MVCC — Contrôle de concurrence multiversion : les lecteurs utilisent des versions stockées adaptées des lignes.

instantané — Versions validées des lignes visibles à un moment défini, plus les modifications propres.

version de ligne — État stocké d’une ligne servant à déterminer sa visibilité.

Read Committed ouvre une nouvelle vue à chaque instruction

Un niveau d’isolement choisit les règles de visibilité et de conflit d’une transaction. READ COMMITTED est le niveau par défaut de PostgreSQL. Un SELECT ordinaire voit les lignes validées avant le début de l’instruction ainsi que les modifications de sa propre transaction. Pour tester une lecture répétée, réinitialisez le produit 1 à cinq et démarrez dans A. Exécutez son premier SELECT, puis faites une pause avant toute modification par B.

DÉMONSTRATION INTERACTIVE / suivez l’ordre des sessions

```
BEGIN ISOLATION LEVEL READ COMMITTED;
SELECT stock FROM work.stock_demo
WHERE product_id = 1;
```

Suivez le raisonnement

1. Réinitialisation : les deux sessions sont hors transaction ; le produit 1 vaut 5.
2. SESSION A, étape 1 : exécutez le bloc affiché ; stock attendu : 5, puis faites une pause.
3. État attendu : A a une transaction ouverte ; B n’a rien modifié.

Sens des mots-clés

niveau d’isolement — Règles de visibilité et de conflit choisies pour une transaction.

READ COMMITTED — Niveau où chaque instruction ordinaire démarre avec une nouvelle vue validée.

ISOLATION LEVEL — Option de BEGIN sélectionnant ces règles.

Valider entre les deux lectures de A

Après la première lecture de A, B fait passer le produit 1 de cinq à quatre et valide. Ce n’est qu’après la fin du COMMIT de B que A répète son SELECT. La transaction de A reste ouverte, mais sa deuxième instruction reçoit un instantané plus récent sous Read Committed. Terminez A par ROLLBACK, car A n’a fait que lire ; cette fin ne peut annuler la modification distincte déjà validée par B.

DÉMONSTRATION INTERACTIVE / suivez l’ordre des sessions

```
BEGIN;
UPDATE work.stock_demo SET stock = stock - 1
WHERE product_id = 1;
COMMIT;
```

Sens des mots-clés

instantané de début d’instruction — Vue fixée au début d’une instruction individuelle.

portée d’une transaction — Travail contrôlé par les limites d’une transaction.

Suivez le raisonnement

1. SESSION B, étape 2 : exécutez le bloc affiché après que A a lu 5 ; B valide stock à 4.

2. SESSION A, étape 3 : répétez son SELECT précédent ; valeur attendue : 4.

3. SESSION A, étape 4 : exécutez ROLLBACK ; les deux transactions sont terminées.

Nommer l’observation : lecture non répétable, pas lecture sale

Une lecture non répétable survient lorsqu’une même transaction relit une ligne et voit une modification validée ultérieurement par une autre transaction. C’est exactement le résultat cinq puis quatre observé dans A. Une lecture sale signifierait voir une modification non validée d’une autre transaction. PostgreSQL interdit les lectures sales : même READ UNCOMMITTED se comporte comme Read Committed. Des résultats différents entre instructions n’impliquent donc pas une fuite de données inachevées.

Résultat / données d’exemple

Démonstration distincte	Première lecture de A ou B	Lectures ultérieures	Signification
B lit alors que A n’a pas validé	5	5 jusqu’à la validation de A	Aucune lecture sale
A relit après la validation de B	5	4	Lecture non répétable autorisée sous RC

Nommer l’observation : lecture non répétable, pas lecture sale / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

lecture non répétable — Relire une ligne renvoie une valeur validée ultérieurement par une autre transaction.

lecture sale — Lire une valeur écrite par une autre transaction avant sa validation.

READ UNCOMMITTED — Niveau demandé que PostgreSQL implémente comme Read Committed.

Suivez le raisonnement

1. État final attendu après l’exécution RC : le produit 1 est validé à 4.

2. Avant l’expérience suivante, terminez les deux transactions et réinitialisez le produit 1 à 5.

Repeatable Read fixe sa vue à la première instruction réelle

REPEATABLE READ conserve un instantané pour les lectures ordinaires après sa première instruction autre que de contrôle de transaction. BEGIN seul ne l’établit pas. Pour prouver ce moment, A démarre à ce niveau et s’arrête avant tout SELECT. B passe de cinq à quatre et valide. Le premier SELECT de A voit ensuite quatre, car cette modification a été validée avant l’établissement de l’instantané de A.

DÉMONSTRATION INTERACTIVE / suivez l’ordre des sessions

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Suivez le raisonnement

1. Réinitialisez le produit 1 à 5 ; les deux sessions doivent être hors transaction.
2. SESSION A, étape 1 : exécutez uniquement le BEGIN affiché et faites une pause.
3. SESSION B, étape 2 : BEGIN; UPDATE work.stock_demo SET stock = 4 WHERE product_id = 1; COMMIT;
4. SESSION A, étape 3 : SELECT stock FROM work.stock_demo WHERE product_id = 1; valeur attendue : 4 ; gardez A ouverte.

Sens des mots-clés

REPEATABLE READ — Niveau conservant un instantané pour les lectures ordinaires suivantes.

établissement de l’instantané — Moment où la vue de visibilité conservée par la transaction est choisie.

Les validations ultérieures ne remplacent pas l’instantané conservé par A

Poursuivez la même expérience après que le premier SELECT de A a renvoyé quatre. B passe maintenant de quatre à trois et valide. Le deuxième SELECT ordinaire de A renvoie toujours quatre sous Repeatable Read. Une nouvelle transaction démarrée après la validation de B voit trois. Le tableau distingue la valeur actuellement validée de la valeur visible par A, montrant pourquoi « validé » et « visible » ne sont pas toujours synonymes.

Résultat / données d’exemple

Moment	Stock validé	SELECT de A
Première validation de B, avant la première lecture de A	4	4
Deuxième validation de B, pendant que A reste ouverte	3	4
Après la fin de A et une nouvelle lecture	3	3

Sens des mots-clés

instantané conservé — Même vue de visibilité établie, réutilisée par les lectures ordinaires suivantes.

nouvelle transaction — Unité distincte pouvant établir un instantané plus récent.

Les validations ultérieures ne remplacent pas l’instantané conservé par A / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. SESSION B, étape 4 : BEGIN; UPDATE work.stock_demo SET stock = 3 WHERE product_id = 1; COMMIT;
2. SESSION A, étape 5 : répétez son SELECT du stock ; valeur attendue : 4.
3. SESSION A, étape 6 : ROLLBACK, puis exécutez le SELECT du stock hors de l’ancienne transaction ; valeur attendue : 3.

Un fantôme est un ensemble modifié de lignes correspondantes

Une lecture fantôme survient lorsqu'une condition réexécutée renvoie un ensemble de lignes différent parce qu'une autre transaction a validé des modifications correspondantes. Ici, la condition est stock > 0 et B insère un troisième produit disponible entre les comptages de A. Read Committed renvoie deux puis trois ; PostgreSQL Repeatable Read renvoie deux puis deux si le premier comptage précède l'insertion de B. Un fantôme concerne l'appartenance à l'ensemble, pas seulement la valeur modifiée d'une ligne.

Résultat / données d'exemple

Réinitialisation séparée	Premier comptage de A	Deuxième comptage de A	Nouveau membre validé
READ COMMITTED	2	3	Produit 3
REPEATABLE READ	2	2	Produit 3, invisible dans l'ancienne vue de A

Sens des mots-clés

lecture fantôme — Une condition répétée correspond à un autre ensemble de lignes après validation d'une autre transaction.

ensemble de lignes correspondantes — Lignes satisfaisant une condition WHERE.

Un fantôme est un ensemble modifié de lignes correspondantes / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. SESSION A, étape 1 : BEGIN au niveau d’isolement choisi ; SELECT count(*) FROM work.stock_demo WHERE stock > 0; valeur attendue : 2.

2. SESSION B, étape 2 : BEGIN; INSERT INTO work.stock_demo VALUES (3, 5); COMMIT;

3. SESSION A, étape 3 : répétez le comptage, puis ROLLBACK. Comparez les résultats attendus ci-dessous.

4. Nettoyage hors transaction : DELETE FROM work.stock_demo WHERE product_id = 3; réinitialisez les produits 1 et 2 à 5.

Une règle entre plusieurs lignes peut échouer malgré des lectures répétables

Certaines règles dépendent de plusieurs lignes ensemble. CampusMarket exige qu’au moins un membre d’une équipe de garde de deux personnes reste de garde. Une anomalie d’écriture survient lorsque des transactions lisent la même situation valide, modifient des lignes différentes et violent ensemble cette règle commune. Préparez cette table de garde distincte. Une contrainte vérifiant une seule ligne à la fois ne peut exprimer toute la règle exigeant au moins une personne.

SQL / lire l’instruction une clause à la fois

```sql
CREATE TABLE IF NOT EXISTS work.duty_roster (
  staff_id integer PRIMARY KEY,
  on_duty boolean NOT NULL
);
TRUNCATE work.duty_roster;
INSERT INTO work.duty_roster
VALUES (1, true), (2, true);
```

Sens des mots-clés

règle entre plusieurs lignes — Condition dépendant de plus d’une ligne.

anomalie d’écriture — Des écritures sur des lignes différentes violent ensemble une règle après lecture d’anciennes données compatibles.

booléen — Valeur logique comme true ou false.

Anomalie d’écriture : les deux décisions utilisent le même ancien comptage

Exécutez les deux sessions sous Repeatable Read. Chacune compte d’abord deux personnes de garde ; chaque application décide donc qu’une personne peut partir. Seulement après les deux comptages, A retire de garde staff 1 et B retire de garde staff 2. Les écritures visent des lignes différentes et n’exigent donc pas le même verrou de ligne. Les deux transactions peuvent valider et ne laisser personne de garde.

DÉMONSTRATION INTERACTIVE / suivez l’ordre des sessions

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT count(*) FROM work.duty_roster
WHERE on_duty;
```

Sens des mots-clés

dépendance de décision — Une écriture est choisie en raison d’une valeur lue auparavant.

anomalie de sérialisation — Résultat concurrent impossible avec une exécution correcte une transaction à la fois.

Anomalie d’écriture : les deux décisions utilisent le même ancien comptage / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. SESSION A, étape 1 : exécutez le bloc affiché et lisez 2 ; faites une pause.
2. SESSION B, étape 2 : exécutez le même bloc et lisez 2 ; faites une pause avant toute écriture.
3. SESSION A, étape 3 : UPDATE work.duty_roster SET on_duty = false WHERE staff_id = 1; COMMIT;
4. SESSION B, étape 4 : UPDATE work.duty_roster SET on_duty = false WHERE staff_id = 2; COMMIT;

Le résultat de l’anomalie d’écriture viole la règle commune

La liste de garde finale contient deux valeurs false et un effectif de garde nul. Chaque transaction a lu un instantané stable et modifié uniquement sa propre ligne, mais le résultat combiné viole la règle d’équipe. Si les décisions correctes s’exécutaient une à la fois, la seconde personne ne verrait plus qu’un collègue et resterait de garde. Des lectures stables ne protègent donc pas à elles seules toutes les règles métier.

Résultat / données d’exemple

Moment	Personnel 1	Personnel 2	Nombre de personnes de garde
Les deux instantanés initiaux	true	true	2
A valide son départ	false	true	1
B valide son départ	false	false	0

Sens des mots-clés

lectures stables — Les lectures ordinaires répétées conservent le même instantané de transaction.

exécution séquentielle — Une transaction complète se termine avant que la suivante commence.

Le résultat de l’anomalie d’écriture viole la règle commune / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. État final attendu : les deux transactions peuvent valider sous Repeatable Read, laissant un effectif de 0.

2. Avant d’essayer Serializable, terminez les deux transactions et réinitialisez toutes les lignes de garde à on_duty = true.

Serializable rejette un historique combiné dangereux

SERIALIZABLE exige que les transactions concurrentes réussies aient un effet possible selon un ordre d’exécution séquentielle. Répétez l’expérience de garde à ce niveau, avec les deux lectures avant toute écriture. PostgreSQL doit empêcher la validation conjointe des deux décisions de départ ; l’une peut échouer avec SQLSTATE 40001. SQLSTATE est un identifiant d’erreur stable à cinq caractères, qu’un programme peut examiner sans dépendre du texte traduit de l’erreur.

DÉMONSTRATION INTERACTIVE / suivez l’ordre des sessions

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT count(*) FROM work.duty_roster
WHERE on_duty;
```

Sens des mots-clés

SERIALIZABLE — Isolement exigeant des résultats réussis équivalents à un ordre d’exécution séquentielle.

SQLSTATE — Identifiant d’erreur de base de données à cinq caractères.

40001 — Le SQLSTATE serialization_failure.

Serializable rejette un historique combiné dangereux / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. Réinitialisez les deux lignes de garde à true. SESSION A, puis SESSION B exécutent chacune ce bloc et lisent 2 avant toute écriture.
2. SESSION A tente de retirer staff 1 de garde ; SESSION B tente de retirer staff 2, avec leurs commandes UPDATE précédentes.
3. Toutes deux tentent COMMIT. Attendez au moins un échec de sérialisation plutôt que deux départs validés.
4. La session en échec exécute ROLLBACK, puis lance une tentative entièrement nouvelle et relit l’effectif.

Exercice 2 : retenter la décision, pas seulement l’écriture

Dans l’expérience de garde Serializable, supposons que A valide le retrait de staff 1 et que B reçoive 40001. L’application de B veut réessayer. Prévoyez le nouvel effectif de garde et décidez si staff 2 peut maintenant partir. Écrivez un plan de nouvelle tentative en quatre étapes avec seulement BEGIN, SELECT, la décision de l’application et une fin de transaction. Expliquez pourquoi recopier l’ancien UPDATE de B dans une nouvelle transaction serait incorrect.

Sens des mots-clés

nouvelle tentative — Lancer une nouvelle tentative après l’échec d’une précédente.

nouvelles tentatives bornées — Politique de nouvelles tentatives avec une limite et une issue finale d’échec.

Suivez le raisonnement

1. Supposez la liste validée : staff 1 false, staff 2 true.
2. Abandonnez l’ancienne décision de B après ROLLBACK.
3. Démarrez une nouvelle transaction Serializable et prévoyez l’effectif.
4. Précisez s’il faut modifier et quand l’application doit cesser de réessayer.

Corrigé expliqué : une nouvelle lecture change la décision

B annule la tentative échouée, démarre une nouvelle transaction Serializable et relit la liste de garde. L’effectif vaut maintenant un : la règle interdit donc un autre départ. B valide une transaction sans modifier la liste et signale que staff 2 doit rester. Ne retentez toute la transaction que pour les échecs appropriés, selon une politique bornée de l’application ; une nouvelle tentative réussie n’effectue pas forcément l’écriture initialement demandée.

Résultat / données d’exemple

Étape de nouvelle tentative	Résultat ou décision
ROLLBACK de la tentative B échouée	Ancien effectif et décision d’écriture abandonnés
BEGIN ISOLATION LEVEL SERIALIZABLE	Nouvelle transaction
SELECT count(*) WHERE on_duty	1
Appliquer la règle d’au moins une personne	Personnel 2 reste ; aucun UPDATE
COMMIT	La liste reste false / true

Corrigé expliqué : une nouvelle lecture change la décision / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

nouvelle tentative — Nouvelle transaction avec de nouvelles lectures et des décisions réévaluées.

attente progressive — Court délai croissant entre les nouvelles tentatives.

Une lecture puis écriture naïve peut perdre une réservation

Une mise à jour perdue survient lorsqu’une application écrase la modification d’une autre transaction avec une valeur périmée lue auparavant. Réinitialisez le produit 1 à cinq. A et B lisent chacun cinq et prévoient chacun de réserver quatre unités. Si tous deux écrivent ensuite la constante calculée un, la seconde écriture remplace le premier résultat au lieu de soustraire une autre réservation. Du SQL valide peut donc enregistrer moins de modifications que promis aux clients.

DÉMONSTRATION INTERACTIVE / suivez l’ordre des sessions

```
SELECT stock FROM work.stock_demo
WHERE product_id = 1;
```

Sens des mots-clés

mise à jour perdue — Une écriture périmée écrase l’effet de la modification d’une autre transaction.

valeur périmée — Valeur lue auparavant qui n’est plus actuelle.

Une lecture puis écriture naïve peut perdre une réservation / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. Réinitialisez le produit 1 à 5. SESSION A et SESSION B exécutent chacune BEGIN sous Read Committed puis ce SELECT ; toutes deux obtiennent 5.

2. Les deux applications calculent 5 - 4 = 1 hors SQL.

3. SESSION A : UPDATE work.stock_demo SET stock = 1 WHERE product_id = 1; COMMIT;

4. SESSION B : UPDATE work.stock_demo SET stock = 1 WHERE product_id = 1; COMMIT;

L’unité finale masque huit unités promises

Le stock final vaut un, mais quatre unités ont été promises à chaque client, soit huit sur les cinq initiales. Le problème est la constante périmée de la seconde écriture, pas une lecture sale. Une contrainte CHECK de non-négativité accepte un et ne peut détecter l’effet métier manquant. Il faut une commande vérifiant la disponibilité actuelle et effectuant la soustraction en une seule opération protégée.

Résultat / données d’exemple

Étape	Stock enregistré	Total des unités promises
État initial	5	0
A écrit 1 et valide	1	4
B écrit la valeur périmée 1 et valide	1	8

Sens des mots-clés

intention métier — Effet réel qu’une commande est censée enregistrer.

écriture d’une constante périmée — Écrire une valeur précalculée sans utiliser la valeur actuellement stockée.

L’unité finale masque huit unités promises / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. État attendu : stock à 1 malgré deux réservations promises de 4 ; ce comportement métier est incorrect.

2. Terminez les deux transactions et réinitialisez le produit 1 à 5 avant l’expérience d’UPDATE protégé.

Utiliser un seul UPDATE arithmétique protégé

L’UPDATE protégé combine le test de disponibilité et la soustraction en une instruction : stock >= 4 n’autorise qu’un produit avec assez d’unités et stock = stock - 4 utilise la valeur stockée de la ligne. UPDATE obtient un verrou de ligne coordonnant les écritures concurrentes. Sous Read Committed, une mise à jour en attente revérifie sa condition sur la version de ligne validée par la première écriture avant de la modifier.

DÉMONSTRATION INTERACTIVE / suivez l’ordre des sessions

```
BEGIN;
UPDATE work.stock_demo
SET stock = stock - 4
WHERE product_id = 1 AND stock >= 4
RETURNING product_id, stock;
```

Sens des mots-clés

UPDATE protégé — Écriture dont la clause WHERE impose la condition requise pour cette écriture.

verrou de ligne — Coordination bloquant les modifications incompatibles de la même ligne.

>= — Supérieur ou égal à.

Suivez le raisonnement

1. Réinitialisez le produit 1 à 5 avec les deux sessions hors transaction.
2. SESSION A, étape 1 : exécutez le bloc affiché ; résultat attendu : (1, 1), puis laissez A ouverte.
3. État attendu : A détient le verrou de ligne ; le stock validé reste 5 jusqu’à la fin de A.

Une réservation en attente peut se terminer sans ligne

B exécute maintenant le même UPDATE protégé et attend, car A détient le verrou de ligne. Après la validation de A, B vérifie la nouvelle quantité de un ; un ne satisfait pas stock >= 4, donc B ne modifie rien et RETURNING est vide. Ce résultat SQL normal signifie que la réservation demandée n’a pas eu lieu. La réussite de A et le refus de B laissent exactement une unité disponible.

Résultat / données d’exemple

Événement chronologique	Résultat de A	Résultat de B	Stock validé
A passe de 5 à 1	(1, 1)	Non exécuté	5
B essaie pendant que A est ouverte	Ouverte	En attente	5
A valide ; B revérifie	Validée	Aucune ligne	1

Sens des mots-clés

revérification de condition — Nouveau test de WHERE sur la ligne cible nouvellement validée après une attente.

RETURNING vide — Aucune ligne n’a été modifiée, donc aucune ligne modifiée ne peut être renvoyée.

Une réservation en attente peut se terminer sans ligne / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. SESSION B, étape 2 : BEGIN ; exécutez le même UPDATE protégé que sur la diapositive précédente et observez son attente.
2. SESSION A, étape 3 : COMMIT ; la réservation renvoyée par A est maintenant acceptée.
3. SESSION B, étape 4 : l’UPDATE se termine sans ligne ; faites ROLLBACK de cette tentative de réservation infructueuse.

Exercice 3 : prévoir une réservation concurrente plus petite

Réinitialisez le produit 1 à cinq et répétez l’expérience d’UPDATE protégé. Cette fois, A demande trois unités et B deux. A effectue son UPDATE en premier et reste ouverte ; B tente son UPDATE protégé pendant que A est ouverte. Prévoyez le comportement immédiat de B, les deux valeurs RETURNING et la quantité finale validée si A et B valident toutes deux. Utilisez l’arithmétique sur les valeurs stockées actuelles plutôt que des constantes calculées par l’application.

Sens des mots-clés

réservation concurrente — Autre transaction tentant de réserver le même produit.

résultat de revérification — Indique si la ligne mise à jour satisfait encore la disponibilité demandée.

Suivez le raisonnement

1. SESSION A modifie le stock avec stock - 3 et stock >= 3, sans encore valider.
2. SESSION B tente stock - 2 avec stock >= 2.
3. Prévoyez ce qui se passe avant et après la validation de A.
4. Écrivez le stock final et le total des unités réellement réservées.

Corrigé expliqué : deux réservations réussies peuvent utiliser les cinq unités

A renvoie un stock de deux après avoir réservé trois unités. B attend d’abord le verrou de ligne de A, puis utilise la valeur validée deux. Sa garde stock >= 2 réussit : son UPDATE arithmétique renvoie donc zéro unité restante. Après validation de B, exactement cinq unités ont été réservées et le stock est nul. Une quantité renvoyée de zéro signifie ici une réussite ; zéro ligne renvoyée signifierait qu’aucune réservation n’a eu lieu.

Résultat / données d’exemple

Étape	Lignes renvoyées	Stock renvoyé	Stock validé ensuite
A réserve 3 ; COMMIT	1	2	2
B reprend et réserve 2 ; COMMIT	1	0	0
Total des réservations acceptées	3 + 2 unités	5 unités	0 restante

Sens des mots-clés

valeur zéro — Ligne renvoyée dont la colonne stock contient le nombre 0.

zéro ligne — Jeu de résultats vide, sans ligne de produit renvoyée.

FOR UPDATE verrouille une ligne avant une décision en plusieurs étapes

SELECT ... FOR UPDATE lit et verrouille les lignes sélectionnées : les écritures concurrentes ou lectures avec verrou doivent attendre la fin de la transaction. Utilisez-le lorsqu'une application a réellement besoin de plusieurs étapes après lecture d'une ligne. Elle doit toujours examiner le stock et décider de continuer. Pour une simple soustraction sur un produit, l'UPDATE protégé est plus court et garde la décision dans l'instruction d'écriture.

DÉMONSTRATION INTERACTIVE / suivez l'ordre des sessions

```
BEGIN;
SELECT product_id, stock
FROM work.stock_demo
WHERE product_id = 1
FOR UPDATE;
```

Sens des mots-clés

FOR UPDATE — Clause SELECT verrouillant les lignes renvoyées contre les écritures et verrouillages incompatibles.

lecture avec verrou — SELECT demandant un verrou de ligne plutôt qu'une simple valeur visible.

FOR UPDATE verrouille une ligne avant une décision en plusieurs étapes / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. Réinitialisez le produit 1 à 5 ; SESSION A exécute le bloc affiché et lit (1, 5).
2. SESSION A fait une pause avec sa transaction ouverte ; elle détient un verrou mais n’a modifié aucun stock.
3. SESSION B suit ensuite les observations distinctes de la diapositive suivante.

Un verrou de ligne coordonne les écritures ; il ne masque pas la ligne

Pendant que A détient son verrou FOR UPDATE, le SELECT ordinaire de B renvoie toujours la quantité validée cinq. La lecture FOR UPDATE ultérieure de B attend la fin de A. A peut alors effectuer une réservation protégée de quatre unités et valider. B reprend sous Read Committed, verrouille la ligne mise à jour et lit un. B doit utiliser cette valeur renvoyée pour sa décision suivante, pas l’ancienne valeur cinq lue sans verrou.

Résultat / données d’exemple

Opération de B pendant que A détient la ligne	Résultat immédiat	Après validation du stock à 1 par A
SELECT ordinaire	5 ; se termine	Aucune requête en cours à modifier
SELECT … FOR UPDATE	Attend	Renvoie 1 et détient son propre verrou

Sens des mots-clés

durée du verrou — Période de détention d’un verrou, normalement jusqu’à la fin de la transaction.

version de ligne mise à jour — Ligne validée plus récente renvoyée après l’attente sous Read Committed.

Un verrou de ligne coordonne les écritures ; il ne masque pas la ligne / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. SESSION B : BEGIN ; exécutez d’abord le SELECT ordinaire du stock et obtenez 5.
2. SESSION B : exécutez le même SELECT avec FOR UPDATE ; il attend.
3. SESSION A : exécutez l’UPDATE protégé de quatre unités, puis COMMIT.
4. SESSION B : son SELECT avec verrou renvoie stock à 1 ; examinez-le, puis ROLLBACK.

Un interblocage est un cycle de transactions s’attendant mutuellement

Un interblocage survient lorsque des transactions détiennent des verrous nécessaires aux autres, créant un cycle d’attente qui ne peut s’achever sans aide. Réinitialisez les deux produits à cinq. A verrouille le produit 1 et B le produit 2. A demande ensuite le produit 2 et attend. Lorsque B demande le produit 1, chacune exige un verrou détenu par l’autre. PostgreSQL détecte le cycle et interrompt un participant.

DÉMONSTRATION INTERACTIVE / suivez l’ordre des sessions

```
UPDATE work.stock_demo SET stock = stock
WHERE product_id = 1;
```

Suivez le raisonnement

1. SESSION A : BEGIN ; exécutez l’UPDATE affiché pour verrouiller le produit 1.
2. SESSION B : BEGIN ; exécutez l’UPDATE équivalent pour product_id = 2.
3. SESSION A : exécutez l’UPDATE pour product_id = 2 ; il attend.
4. SESSION B : exécutez l’UPDATE pour product_id = 1 ; un cycle se forme et un côté reçoit 40P01. Les deux terminent par ROLLBACK.

Sens des mots-clés

interblocage — Cycle d’attente de verrous où chaque participant exige un verrou détenu par un autre.

mise à jour sans effet — UPDATE attribuant la valeur existante, tout en acquérant des verrous.

40P01 — Le SQLSTATE deadlock_detected.

Voir le cycle d’attente créant un interblocage

Un interblocage est un cycle d’attente, pas simplement une requête lente. La session A détient déjà un verrou nécessaire à B et B détient un verrou nécessaire à A. Aucune ne peut terminer sans intervention. PostgreSQL détecte le cycle et interrompt une transaction. Acquérir les mêmes lignes requises dans un ordre cohérent empêche ce cas précis.



Les deux attentes forment un cycle. PostgreSQL interrompt une transaction pour permettre à l’autre de continuer.

Utiliser un ordre de verrouillage cohérent pour éviter le cycle montré

L’interblocage comporte deux dépendances : A attend le verrou de B sur le produit 2 et B attend celui de A sur le produit 1. Si chaque transaction acquiert les produits par identifiant croissant, B attend sur le produit 1 avant de détenir le produit 2. Cela supprime ce cycle. Gardez les transactions courtes et évitez d’attendre une saisie utilisateur avec des verrous détenus ; l’ordre réduit les interblocages mais ne remplace pas la gestion des erreurs.

Résultat / données d’exemple

Ordonnancement	A détient / attend	B détient / attend	Résultat
Ordre opposé	Détient 1 ; attend 2	Détient 2 ; attend 1	Cycle d’interblocage
Les deux demandent 1 puis 2	Détient 1, puis 2	Attend 1 ; ne détient aucun des deux	A termine ; B poursuit ensuite

Sens des mots-clés

cycle d’attente — Chaîne fermée où chaque transaction attend une autre transaction de la chaîne.

ordre cohérent — Toutes les transactions participantes demandent les ressources partagées dans le même ordre.

Utiliser un ordre de verrouillage cohérent pour éviter le cycle montré / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. Pour la nouvelle exécution ordonnée, les deux sessions demandent le produit 1 avant le produit 2.
2. État attendu : l’une peut attendre, mais ce cycle à deux ressources ne peut se former ; terminez les deux transactions.

Les délais limites bornent l’attente, pas la correction d’une réservation

Un délai limite borne le temps d’attente ou d’exécution d’une opération avant son annulation par PostgreSQL. lock_timeout limite chaque attente d’acquisition de verrou ; statement_timeout limite toute l’instruction, attente comprise. SET LOCAL n’applique un paramètre que dans la transaction courante. Ces limites préservent la réactivité d’une application, mais une instruction annulée exige toujours une gestion d’erreur et une décision sur toute la transaction métier.

SQL / lire l’instruction une clause à la fois

```sql
BEGIN;
SET LOCAL lock_timeout = '3s';
SET LOCAL statement_timeout = '10s';
SELECT stock FROM work.stock_demo
WHERE product_id = 1 FOR UPDATE;
ROLLBACK;
```

Sens des mots-clés

lock_timeout — Attente maximale pour chaque acquisition de verrou.

statement_timeout — Durée maximale autorisée pour toute une instruction.

SET LOCAL — Définit une option uniquement jusqu’à la fin de la transaction courante.

Lire le SQLSTATE initial avant de décider de réessayer

Les codes d’erreur indiquent à l’application ce qui a échoué, sans rendre toute nouvelle tentative sûre. Les échecs de sérialisation et interblocages exigent normalement une nouvelle transaction entière avec de nouvelles lectures et décisions. Les délais de verrou ou d’instruction exigent une réponse propre à l’application et une annulation dans ce modèle simple. Après une erreur, 25P02 indique seulement que la transaction est déjà en échec ; cherchez l’erreur antérieure à l’origine de cet état.

Résultat / données d’exemple

SQLSTATE	Signification	Réponse simple à appliquer
40001	serialization_failure	ROLLBACK ; nouvelles tentatives bornées de toute la transaction
40P01	deadlock_detected	ROLLBACK ; nouvelles tentatives bornées de toute la transaction
55P03	lock_not_available, y compris dépassement du délai de verrou	ROLLBACK ; gérer la contention selon la politique de l’application
57014	query_canceled, y compris dépassement du délai d’instruction	ROLLBACK ; examiner ou appliquer une politique choisie
25P02	Transaction déjà en échec	Trouver l’erreur initiale ; ROLLBACK avant tout nouveau travail

Lire le SQLSTATE initial avant de décider de réessayer / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

périmètre de nouvelle tentative — Toutes les lectures, décisions et écritures à réexécuter ensemble.

25P02 — in_failed_sql_transaction : il faut d’abord récupérer après l’échec antérieur.

issue inconnue — Le client ne peut déterminer si une validation s’est achevée.

Une clé de demande unique rend une insertion répétée détectable

Une clé unique identifie une demande qui ne doit pas être enregistrée deux fois. Dans work.request_log, la clé primaire est request_key : deux lignes ne peuvent donc partager reservation-42. INSERT ... ON CONFLICT nomme cette règle d’unicité et choisit l’action en cas de conflit. DO NOTHING ignore l’insertion en doublon ; RETURNING n’inclut que les lignes réellement insérées. Cela fournit un élément de base pour reconnaître les demandes clientes répétées.

SQL / lire l’instruction une clause à la fois

```sql
CREATE TABLE IF NOT EXISTS work.request_log (
  request_key text PRIMARY KEY,
  result text NOT NULL
);
TRUNCATE work.request_log;
INSERT INTO work.request_log
VALUES ('reservation-42', 'reserved')
ON CONFLICT (request_key) DO NOTHING
RETURNING request_key, result;
```

Sens des mots-clés

clé unique — Valeurs de colonnes qui ne peuvent apparaître dans deux lignes.

ON CONFLICT — Choisit une alternative lorsqu’une insertion viole la règle d’unicité nommée.

DO NOTHING — Ignore une insertion en conflit sans insérer cette ligne.

excluded désigne la ligne proposée dans un upsert

Un upsert insère une nouvelle ligne ou met à jour la ligne existante en conflit. ON CONFLICT … DO UPDATE nomme cette branche de mise à jour. Le nom spécial excluded désigne les valeurs proposées par l’insertion, tandis qu’un alias comme existing désigne la ligne stockée. Ici, un import de libellés de stock remplace le libellé d’un produit en cas de conflit. Il ne soustrait pas de stock, ne débite pas d’argent et ne répète pas l’effet d’une réservation.

SQL / lire l’instruction une clause à la fois

```sql
CREATE TABLE IF NOT EXISTS work.product_label (
  product_id integer PRIMARY KEY, label text
);
TRUNCATE work.product_label;
INSERT INTO work.product_label
VALUES (1, 'Old label');
INSERT INTO work.product_label AS existing
VALUES (1, 'New label')
ON CONFLICT (product_id) DO UPDATE
SET label = excluded.label
RETURNING product_id, label;
```

excluded désigne la ligne proposée dans un upsert / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

upsert — Insérer une ligne ou mettre à jour la ligne existante correspondant à sa clé unique.

excluded — Ligne proposée à l’insertion, disponible pour la mise à jour sur conflit.

DO UPDATE SET — Attribue des valeurs à la ligne stockée en conflit.

Résultats de conflits commentés : lignes ignorées et mises à jour diffèrent

Répéter l’insertion du journal des demandes renvoie un jeu de lignes vide, car sa clé existe déjà ; la ligne initiale reste accessible par SELECT. L’upsert de libellé renvoie une ligne contenant New label, car la branche de conflit l’a mise à jour. Un doublon ignoré et une mise à jour réussie sont deux résultats différents. Avant d’exécuter l’une de ces commandes dans une application, décidez quel résultat correspond à l’action métier voulue.

Répéter l’INSERT du journal des demandes de D4-REQUEST

Exécution	RETURNING	Ligne request_log stockée
Première exécution	reservation-42 | reserved	Une ligne
Deuxième exécution identique	Aucune ligne	La même ligne unique

Import de libellé de D4-UPSERT

product_id	label
1	New label

Résultats de conflits commentés : lignes ignorées et mises à jour diffèrent / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

demande en doublon — Envoi ultérieur utilisant la même clé de demande.

branche de conflit — Action alternative choisie pour un conflit de clé unique.

MERGE rapproche une source d’une cible avant de choisir les actions

MERGE compare les lignes sources aux lignes cibles avec ON, puis choisit UPDATE ou INSERT au moyen des clauses WHEN. L’alias source incoming fournit un libellé ; l’alias cible stored nomme la table existante. MATCHED signifie que ON a trouvé une cible ; NOT MATCHED signifie le contraire. Cet exemple attribue Latest label au produit 1. Contrairement à ON CONFLICT, une insertion concurrente en doublon peut provoquer une erreur d’unicité au lieu de prendre une branche de mise à jour.

SQL / lire l’instruction une clause à la fois

```sql
MERGE INTO work.product_label AS stored
USING (VALUES (1, 'Latest label'))
  AS incoming(product_id, label)
ON stored.product_id = incoming.product_id
WHEN MATCHED THEN
  UPDATE SET label = incoming.label
WHEN NOT MATCHED THEN
  INSERT (product_id, label)
  VALUES (incoming.product_id, incoming.label);
```

Sens des mots-clés

MERGE — Rapproche une source d’une cible et exécute l’action WHEN sélectionnée.

USING — Introduit les lignes sources de MERGE.

MATCHED — La ligne source a trouvé une cible satisfaisant ON.

NOT MATCHED — La ligne source n’a trouvé aucune cible satisfaisant ON.

Les valeurs d’identité identifient les lignes ; elles ne comptent pas les validations / contexte et définitions

Une colonne d’identité génère des identifiants avec une séquence interne, un générateur de nombres qui avance indépendamment des annulations de transaction. Si une insertion obtient l’identifiant 1 puis est annulée, la suivante peut recevoir 2 même s’il ne reste qu’une ligne. Les sauts sont normaux et les insertions en conflit peuvent aussi consommer des nombres. Utilisez une clé primaire pour l’unicité ; n’interprétez pas des identifiants consécutifs comme preuve d’un historique métier complet.

Sens des mots-clés

colonne d’identité — Colonne dont l’identifiant par défaut provient d’une séquence interne.

séquence — Générateur de nombres dont les valeurs allouées ne sont pas récupérées par une annulation.

DEFAULT VALUES — Insère une ligne avec les valeurs par défaut des colonnes.

Les valeurs d’identité identifient les lignes ; elles ne comptent pas les validations

Montrez pourquoi une insertion annulée peut laisser un saut dans les identifiants générés.

SQL / lire l’instruction une clause à la fois

```sql
BEGIN;
DROP TABLE IF EXISTS work.id_demo;
CREATE TABLE work.id_demo (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY
);
COMMIT;
BEGIN;
INSERT INTO work.id_demo DEFAULT VALUES
RETURNING id;
ROLLBACK;
INSERT INTO work.id_demo DEFAULT VALUES
RETURNING id;
```

L’idempotence empêche les nouvelles tentatives de répéter l’effet

Une opération est idempotente si répéter la même demande a le même effet métier que l’exécuter une fois. Une réservation fiable s’attribue une clé de demande unique, modifie le stock et stocke son résultat dans la même transaction de base de données ; une répétition récupère ce résultat après vérification de la correspondance du contenu de la demande. Un service de paiement externe est hors du périmètre d’annulation de PostgreSQL. Coordonnez sa propre clé d’idempotence et résolvez les issues incertaines au lieu de débiter de nouveau à l’aveugle.

Résultat / données d’exemple

Action répétée ou échouée	Traitement requis
Même clé de réservation et même contenu	Renvoyer le résultat déjà stocké ; ne pas décrémenter de nouveau
Même clé avec un panier différent	Rejeter la différence plutôt que réutiliser un résultat sans rapport
La transaction de réservation en base échoue	Annuler ensemble l’attribution de sa clé, les modifications du stock et le résultat
Paiement accepté, mais réponse au client perdue	Interroger ou réessayer selon les règles d’idempotence du prestataire ; rapprocher le résultat

L’idempotence empêche les nouvelles tentatives de répéter l’effet / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

idempotent — Répéter la même demande ne duplique pas son effet métier.

contenu de la demande — Détails significatifs d’une demande, comme les produits et quantités.

périmètre externe — Travail d’un autre système que l’annulation de cette base ne peut défaire.